

MANUAL TÉCNICO DE DESARROLLO WEB CON DJANGO

Fundamentos de Arquitectura, ORM y Patrón MVT

PARTE I

Módulo de Formación:

Construcción de Software Backend y Frontend

Programa:

Análisis y Desarrollo de Software (ADSO)

Ficha 3196477

Elaborado por: Ing. Antony Reynel Botello Herrera

Instructor Técnico

Servicio Nacional de Aprendizaje - SENA

Contenido

1	Introducción:	6
2	Configuración de Django en VS Code (Parte 1)	8
2.1	Preparación del Entorno en Visual Studio Code	8
2.2	Creación del Entorno Virtual (VENV)	9
2.3	Instalación de Django	11
2.4	Creación del Proyecto con Estructura Personalizada	11
2.5	Resultado de la Estructura de Archivos	12
2.6	Configuración Regional del Proyecto (settings.py)	13
2.7	¡Encendiendo los motores! (Comprobación del Servidor Local)	14
3	Arquitectura Base y el "Ciclo" de Django (Parte 2)	15
3.1	Configuración de Plantillas y Estáticos en settings.py	15
3.2	Creando la Estructura Física de Carpetas	16
3.3	Creación y Registro de nuestra primera Aplicación (App)	17
3.3.1	Concepto Clave: ¿Qué es la Modularidad y por qué es importante?	17
3.3.2	¿Por qué es vital que programemos de forma modular?	18
3.4	Completando la estructura interna del Módulo	19
3.4.1	Organización Profesional de Plantillas (Namespacing)	21
3.5	El Enrutador (URLs) y la función <code>include</code>	23
3.5.1	Configurar las rutas locales de la App (<code>inventario/urls.py</code>)	23

3.5.2	Conectar la App al Proyecto Principal (<code>core/urls.py</code>).....	24
3.6	Preparando la Interfaz (Herencia Dual y Bootstrap).....	25
3.6.1	Crear la Plantilla Pública (<code>base_cliente.html</code>)	26
3.6.2	Crear la Plantilla Administrativa (<code>base_admin.html</code>).....	27
3.7	Crear el Template del Inventario (Usando Clases de Bootstrap)	29
3.8	La Lógica (Nuestra primera Función en la Vista)	30
3.8.1	Concepto Avanzado: Funciones vs. Clases	30
3.8.2	La Anatomía de la Vista y la Introducción al ORM	31
3.9	El puente de comunicación: El Contexto y el Motor de Plantillas	34
3.9.1	¿Qué es el Contexto?	34
3.9.2	Sintaxis del Motor de Plantillas (DTL)	34
3.9.3	Tabla: Uso de Variables y Lógica en el Template	35
3.9.4	Tabla: Filtros de Plantilla más utilizados en Proyectos Web	35
3.10	Creación de nuestro primer Modelo (Base de Datos)	36
3.10.1	Concepto Clave: ¿Qué son los Modelos?	37
3.10.2	Entendiendo las Relaciones en Django	38
3.10.3	Tabla de Referencia: Tipos de Datos para el MER	40
3.11	Preparando y aplicando Migraciones	40
3.12	El Panel de Administración (Nuestra herramienta de Desarrollo)	41
3.12.1	Crear el Superusuario (El SysAdmin)	42

3.12.2	Registrar los Modelos en <code>admin.py</code>	42
3.12.3	Poblar la Base de Datos (Primeros Datos de Prueba)	43
3.13	Conectando la Base de Datos con la Interfaz (Nuestra primera consulta)	43
4	Anexo: ¿Qué es el ORM?	47
4.1	Filtros Avanzados (El uso del Doble Guion Bajo <code>__</code>)	48
4.2	Operaciones Básicas con el ORM (El famoso CRUD)	50
4.3	Filtros con Múltiples Condiciones (AND / OR)	51
4.3.1	El operador "Y" (AND)	51
4.3.2	El operador "O" (OR)	51
4.3.3	Operadores Lógicos en el ORM	52
5	Anexo: Comandos Principales de Django	53
6	Anexo: Solución a Errores Frecuentes (Troubleshooting)	54
6.1	Error 1: "It is impossible to add a non-nullable field... without specifying a default"	54
6.2	Error 2: <code>TemplateDoesNotExist</code> at <code>/ruta/</code>	54
6.3	Error 3: <code>OperationalError: no such table: inventario_producto</code>	55
6.4	Error 4: <code>ModuleNotFoundError: No module named 'django'</code> ..	56

1 Introducción:

Antes de escribir nuestra primera línea de código, es crucial entender con qué herramienta vamos a trabajar y por qué la hemos elegido para nuestro proyecto formativo.

¿Qué es Django y de dónde viene? Django es un framework (marco de trabajo) de desarrollo web de alto nivel, escrito en el lenguaje de programación Python. Nació en el año 2003 en la redacción del periódico *Lawrence Journal-World* en Estados Unidos. Los desarrolladores de este medio necesitaban crear aplicaciones web complejas, seguras y, sobre todo, **muy rápido** para cumplir con los tiempos de las noticias.

En 2005, decidieron liberar su código al público (Open Source) y lo bautizaron "Django" en honor al famoso guitarrista de jazz Django Reinhardt. Su filosofía principal se resume en dos conceptos:

1. **DRY (Don't Repeat Yourself - No te repitas):** Evitar escribir el mismo código varias veces.
2. **"Baterías incluidas":** Viene con todo lo necesario para arrancar, sin tener que inventar la rueda.

¿Por qué usamos Django y no alternativas como Flask?

En el ecosistema de Python, Flask es otra herramienta muy popular. Sin embargo, existe una diferencia técnica clave por la cual optamos por Django para la construcción de nuestro software:

- **Flask es un "Micro-framework":** Es excelente, pero viene casi vacío. Te da la libertad de elegir tu propia base de datos, tu propio sistema de usuarios y tu

propia seguridad. El problema es que, en aplicaciones empresariales, ensamblar todas estas piezas manualmente toma mucho tiempo y aumenta el riesgo de cometer errores de seguridad.

- **Django es un framework completo ("Full-Stack"):** Django ya trae integrado un ORM (Mapeador Objeto-Relacional) para interactuar con la base de datos sin escribir SQL puro, un panel de administración listo para usar, un sistema robusto de autenticación de usuarios y protección automática contra los ataques informáticos más comunes.

Para nuestro objetivo de construir aplicativos web que gestionen información de manera segura, escalable y bajo estándares de la industria, Django nos proporciona una arquitectura sólida (basada en el patrón Modelo-Vista-Template) que nos permite enfocarnos en la lógica del negocio en lugar de perder tiempo configurando la infraestructura base.

2 Configuración de Django en VS Code (Parte 1)

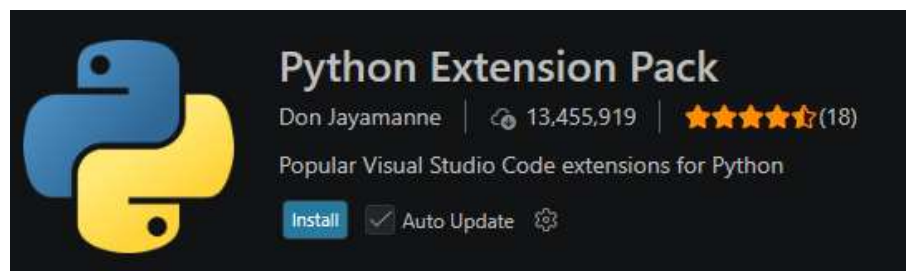
Esta fase inicial se centra en preparar el entorno de desarrollo y la arquitectura base del proyecto, asegurando que el código sea organizado y profesional.

2.1 Preparación del Entorno en Visual Studio Code

Antes de escribir código, debemos asegurar que las herramientas de apoyo estén listas:

1. Instalación de Extensiones:

- Python Extension Pack



- Python Environments



2. Creación del Directorio Raíz: Crea una carpeta física en tu equipo llamada mi-proyecto y ábrela con VS Code.

2.2 Creación del Entorno Virtual (VENV)

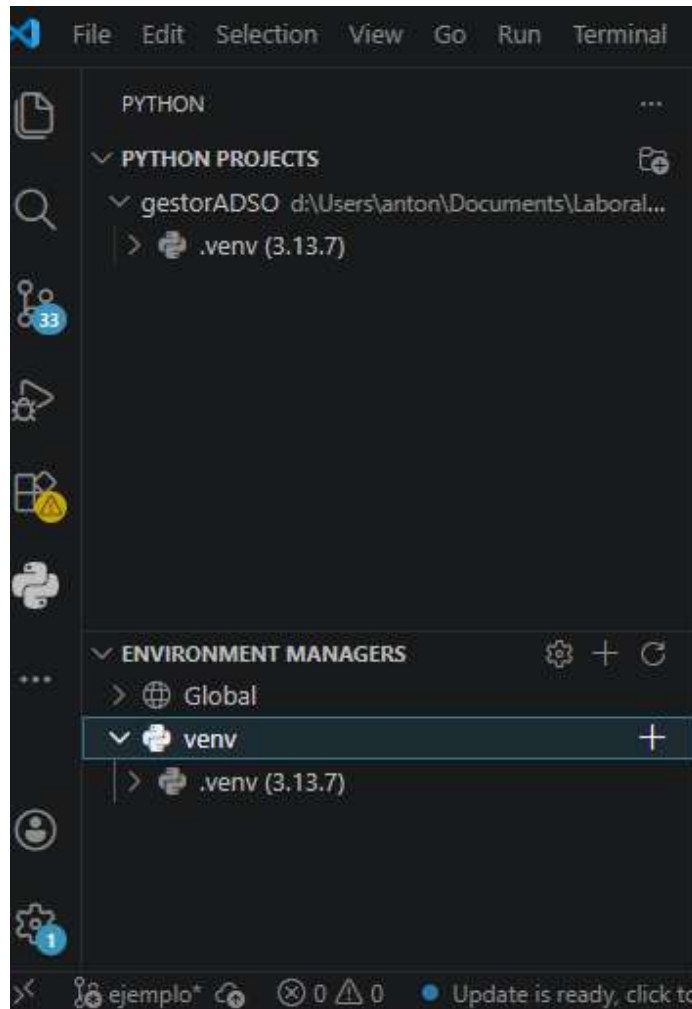
Para garantizar la estabilidad del software y evitar conflictos entre las librerías de diferentes proyectos, es una **buena práctica técnica** trabajar siempre en un entorno aislado. En lugar de usar comandos, aprovecharemos la extensión **Python**

Environment Manager de Visual Studio Code:

1. **Acceso al Gestor:** En la barra de actividad (izquierda), selecciona el icono de **Python**.



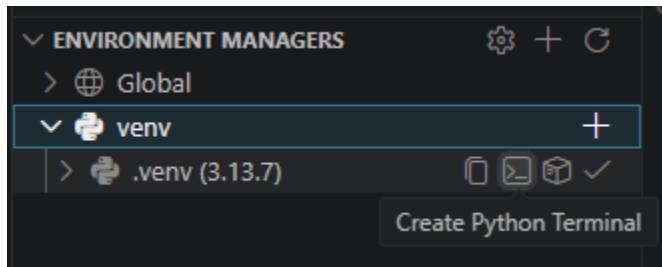
2. **Creación del Entorno:** * Busca la sección **Environment Managers**.



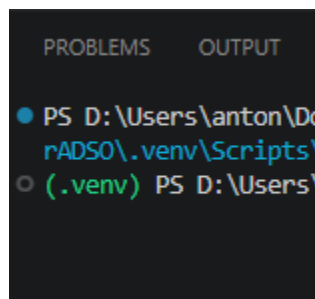
- Al pasar el cursor sobre el título del área, aparecerán tres iconos a la derecha .
- Haz clic en el icono **"+" (Create Environment)** y sigue los pasos del asistente para generar tu entorno virtual.

3. **Identificación:** Una vez finalizado el proceso, verás el entorno listado en ese mismo espacio (comúnmente llamado `.venv` o `venv`).

4. **Activación Automática:** Localiza el nombre de tu entorno, ubica el puntero sobre él y haz clic en el icono de la **Consola (Terminal)** .



Resultado esperado: Se abrirá una terminal integrada en la parte inferior con el entorno ya seleccionado y activado automáticamente, lista para comenzar la instalación de las dependencias de tu proyecto.



2.3 Instalación de Django

Con el entorno activo, procedemos a instalar el framework:

- Ejecuta: `pip install django`
- Para verificar la instalación, puedes usar: `django-admin --version`

2.4 Creación del Proyecto con Estructura Personalizada

Para mantener una arquitectura de software organizada y profesional, configuraremos el proyecto de modo que los archivos principales de configuración y administración residan en una única carpeta central llamada `core`.

1. Verifica que tu consola esté ubicada dentro de la carpeta raíz (`mi-proyecto`) y que el entorno virtual se encuentre activo.

2. Ejecuta el siguiente comando para inicializar el framework:

```
django-admin startproject core .
```

¡Atención al detalle! Es indispensable incluir el espacio y el punto (`.`) al final de la instrucción. Esto le indica a Django que construya la estructura directamente en el directorio actual, evitando que el sistema genere una subcarpeta redundante que desorganice el proyecto.

2.5 Resultado de la Estructura de Archivos

Si los pasos anteriores se ejecutaron correctamente, tu explorador de archivos en VS Code debería tener exactamente la siguiente estructura:

```
mi-proyecto/          <-- (Carpeta Raíz / Principal)
├── core/              <-- (Carpeta de configuración central)
│   ├── __init__.py    <-- (Indica que esta carpeta es un paquete de Python)
│   ├── settings.py    <-- (Ajustes y configuración general del proyecto)
│   ├── urls.py        <-- (Enrutador principal de las URLs)
│   ├── asgi.py        <-- (Configuración para despliegue web asíncrono)
│   └── wsgi.py        <-- (Configuración para despliegue web tradicional)
├── .venv/            <-- (Tu entorno virtual aislado)
└── manage.py         <-- (Utilidad principal de administración de Django)
```

💡 **Tip para el desarrollador:** Piensa en el archivo `manage.py` como el "**control remoto**" de nuestro proyecto. A partir de este momento, utilizaremos este archivo a través de la terminal para casi todas las interacciones importantes: encender el servidor local, administrar la base de datos y crear nuevos módulos.

2.6 Configuración Regional del Proyecto (settings.py)

Por defecto, Django se configura en idioma inglés y con la zona horaria universal (UTC). Para que nuestro software sea pertinente a nuestro contexto y maneje las fechas, horas y mensajes del sistema de manera correcta, debemos ajustar estos valores.

1. En tu explorador de archivos de VS Code, despliega la carpeta core y abre el archivo settings.py.
2. Desplázate casi hasta el final del archivo, donde encontrarás una sección con variables en mayúsculas como LANGUAGE_CODE y TIME_ZONE.
3. Modifica los valores para establecer el español de Colombia y nuestra zona horaria local. Debe quedar exactamente así:

```
LANGUAGE_CODE = 'es-co'  
TIME_ZONE = 'America/Bogota'  
USE_I18N = True  
USE_TZ = True
```

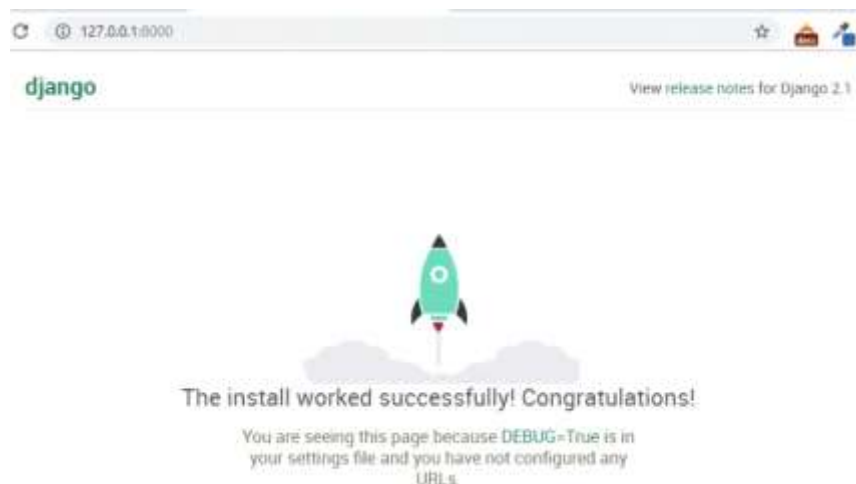
 **Nota técnica:** Al guardar estos cambios (`Ctrl + S`), el panel de administración que Django trae incorporado y los formatos de fecha que guardemos en la base de datos se adaptarán automáticamente a nuestro idioma y región.

2.7 ¡Encendiendo los motores! (Comprobación del Servidor Local)

Es momento de utilizar nuestro "control remoto" (manage.py) para verificar que el entorno, la instalación y la configuración inicial están perfectos.

1. Asegúrate de estar en la terminal, dentro de la carpeta raíz (mi-proyecto) y con tu entorno virtual activo (venv).
2. Ejecuta el comando para levantar el servidor de desarrollo:

```
python manage.py runserver
```
3. Si todo está correcto, la terminal mostrará un mensaje indicando que el servidor ha iniciado y verás una dirección web local, generalmente `http://127.0.0.1:8000/`.
4. Mantén presionada la tecla **Ctrl** y haz clic sobre ese enlace (o cópialo y pégalo en tu navegador web).



3 Arquitectura Base y el "Ciclo" de Django (Parte 2)

Antes de crear nuestra primera aplicación, vamos a preparar el proyecto para que sepa exactamente dónde guardar y buscar nuestros diseños (archivos HTML) y nuestros recursos estáticos (CSS, JavaScript, imágenes).

3.1 Configuración de Plantillas y Estáticos en settings.py

Abre nuevamente el archivo settings.py ubicado dentro de tu carpeta core.

Para las Plantillas (Templates): Busca la lista llamada TEMPLATES. Dentro de ella, hay una clave llamada 'DIRS'. Vamos a indicarle a Django que busque una carpeta global llamada templates en la raíz del proyecto. Modifícalo para que quede así:

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'], # <-- Agregamos
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    ],
]
```

Para los Archivos Estáticos: Desplázate hasta el final del archivo, justo debajo de donde configuramos el idioma. Verás una variable llamada `STATIC_URL`. Debajo de ella, agregaremos la ruta global donde Django buscará nuestros estilos e imágenes. Debe quedar así:

```
STATIC_URL = 'static/'

# Agregamos esta configuración para los estáticos globales
STATICFILES_DIRS = [
    BASE_DIR / 'static',
]
```

💡 **Tip para el desarrollador:** La variable `BASE_DIR` es una herramienta

automática de Django que detecta la ruta exacta de tu proyecto en tu computadora. Al usar `BASE_DIR / 'templates'`, le estamos diciendo: *"Ve a la carpeta raíz y busca la carpeta templates"*, sin importar en qué computador estemos trabajando.

3.2 Creando la Estructura Física de Carpetas

Ahora que Django sabe dónde buscar, debemos crear esas carpetas físicamente para mantener todo organizado y modular.

1. En el explorador de VS Code, haz clic en un espacio vacío para asegurarte de estar en la **carpeta raíz** (mi-proyecto), al mismo nivel que el archivo `manage.py`.
2. Crea una nueva carpeta llamada `templates`.
3. Dentro de `templates`, crea una subcarpeta llamada `partials`. *(Aquí guardaremos fragmentos de código HTML que se repiten, como la barra de navegación o el pie de página).*
4. Vuelve a la raíz y crea otra carpeta llamada `static`.
5. Dentro de `static`, puedes crear subcarpetas para `css`, `js` e `img`.

```

mi-proyecto/
├── core/                <-- (Configuración)
├── static/              <-- (Recursos estáticos globales)
│   ├── css/
│   ├── js/
│   └── img/
├── templates/           <-- (Diseños HTML globales)
│   └── partials/        <-- (Fragmentos reutilizables: navbar, footer)
├── .venv/               <-- (Entorno virtual)
└── manage.py

```

3.3 Creación y Registro de nuestra primera Aplicación (App)

En Django, un **Proyecto** es el sitio web completo (la casa), mientras que una **App** es un módulo específico que cumple una función particular (las habitaciones). Como nuestro objetivo es crear un sistema de gestión, nuestra primera App será el módulo de inventario.

3.3.1 Concepto Clave: ¿Qué es la Modularidad y por qué es importante?

Cuando mencionamos que Django es un framework **modular**, nos referimos a uno de los principios más importantes en el desarrollo de software moderno: la **Modularidad**.

La modularidad es la práctica de dividir un sistema grande y complejo (el Proyecto) en piezas más pequeñas, independientes y manejables (las Apps o Módulos).

Retomando la analogía de la casa:

- **El Proyecto (La Casa):** Es el contenedor principal. Tiene los cimientos y los servicios generales (la configuración en core, la conexión a la base de datos).

- **Las Apps (Las Habitaciones):** Cada habitación tiene un propósito específico. La cocina es para cocinar (Módulo de Inventario), la habitación es para dormir (Módulo de Usuarios), y la sala es para recibir visitas (Módulo de Ventas).

3.3.2 *¿Por qué es vital que programemos de forma modular?*

1. **Trabajo en equipo simultáneo:** En un entorno laboral real, ustedes no trabajarán solos. Si el software es modular, un aprendiz puede estar programando la app de inventario mientras otro compañero codifica la app de usuarios, sin que sus códigos choquen o se dañen entre sí.
2. **Mantenimiento y aislamiento de errores:** Si el día de mañana ocurre un error al registrar un producto, sabremos exactamente que el problema está en la app de inventario. Además, si un módulo falla, no necesariamente tumba todo el sistema.
3. **Reutilización de código (Filosofía DRY):** Si construimos una app de facturación muy bien estructurada para este proyecto, y el próximo año nos contratan para otro desarrollo diferente, ¡podemos llevarnos esa misma carpeta (app) y conectarla al nuevo proyecto casi sin hacer cambios!

Al crear nuestra app inventario, estamos garantizando que toda la lógica de los productos, categorías y existencias viva en su propio espacio, manteniendo nuestro código limpio y profesional.

Ahora volviendo a el paso a paso para crear la primera aplicación.

1. Asegúrate de estar en la terminal, en la raíz del proyecto y con tu entorno virtual activo.
2. Ejecuta el siguiente comando para crear el módulo: `python manage.py startapp inventario`
3. Verás que se ha creado una nueva carpeta llamada inventario con varios archivos de Python (views.py, models.py, etc.).
4. **¡Paso Crítico! (Registro):** Django no sabe que este módulo existe hasta que lo registremos. Abre el archivo settings.py (en la carpeta core), busca la lista `INSTALLED_APPS` y agrega el nombre de tu nueva app al final:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    # Nuestras Apps  
    'inventario', # <-- Registramos nuestro nuevo módulo  
]
```

3.4 Completando la estructura interna del Módulo

Cuando ejecutamos el comando `startapp`, Django nos entrega los archivos básicos para trabajar (como `views.py` y `models.py`). Sin embargo, para cumplir al 100% con el principio de **modularidad** que vimos anteriormente, debemos crear manualmente algunos archivos y carpetas para que este módulo sea totalmente independiente.

Dentro de tu nueva carpeta `inventario`, haz clic derecho y crea lo siguiente:

1. **Un archivo llamado `urls.py`:** * *¿Para qué sirve?* Por defecto, Django solo crea un `urls.py` global en la carpeta `core`. Si ponemos todas las rutas allí, el archivo se volverá un desastre inmanejable. Al crear un `urls.py` local, logramos que la app de inventario administre sus propias direcciones web.
2. **Un archivo llamado `forms.py`:**
 - *¿Para qué sirve?* Aquí construiremos los formularios de manera segura (por ejemplo, el formulario para "Registrar un Nuevo Producto"). Django tiene un sistema muy potente para validar que los usuarios no ingresen datos dañinos, y la buena práctica es separar esa lógica en este archivo.
3. **La estructura de Plantillas (Templates):**
 - Crea una carpeta llamada `templates`.
 - **¡Muy importante!** Dentro de esa carpeta `templates`, crea otra carpeta con el mismo nombre de la app: `inventario`.
 - *¿Para qué sirve?* Aquí guardaremos los archivos HTML de este módulo. Hacer la subcarpeta evita que Django se confunda si el día de mañana tenemos otra app que también tenga un archivo llamado `index.html`.

3.4.1 Organización Profesional de Plantillas (Namespacing)

Al crear la carpeta `templates` dentro de nuestro módulo, debemos aplicar una regla estricta de arquitectura de software en Django: **siempre crear una subcarpeta adicional con el mismo nombre de la app.**

Adicional a esto, agruparemos los archivos según los Modelos (tablas) que vayamos a manejar en la base de datos.

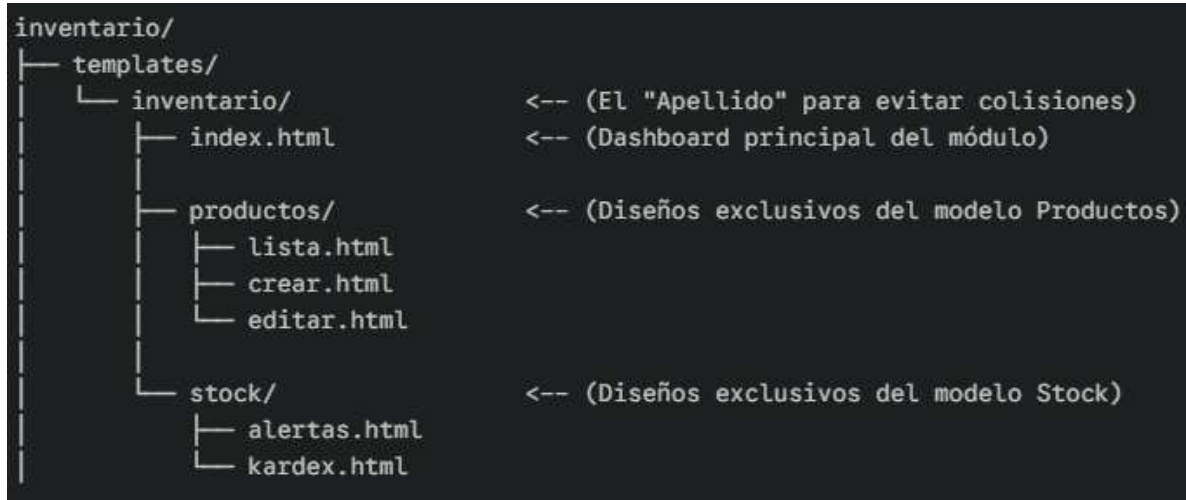
¿Por qué es obligatorio hacer esto? (El problema de la colisión) Cuando Django enciende el servidor, no lee las carpetas de diseño de forma aislada. Lo que hace es buscar todas las carpetas `templates` de todas tus Apps y meter todos los archivos HTML en una sola "bolsa virtual" compartida.

Imagina que tienes el módulo de `inventario` y otro módulo de `usuarios`. Si en ambos creas un archivo llamado `index.html` suelto, Django sufrirá una "colisión de nombres". Al pedirle que cargue el `index.html`, simplemente tomará el primero que encuentre en la bolsa. ¡Podrías terminar mostrando la pantalla de usuarios en la sección de inventario!

La Solución (Namespacing): Para evitar que el sistema se confunda, encapsulamos los diseños en una carpeta con el nombre de la app (esto actúa como un "apellido" único). Luego, subdividimos por modelos para mantener el orden lógico.

1. Dentro de la carpeta `inventario`, ubica tu nueva carpeta `templates`.
2. Adentro, crea una carpeta llamada `inventario`.
3. Dentro de esa carpeta `inventario`, crea subcarpetas para las entidades que vas a manejar, por ejemplo: `productos` y `stock`.

Verificación de la estructura ideal: Tu explorador en VS Code debería verse exactamente así:



💡 **Tip de Arquitectura:** A partir de ahora, cuando en tu archivo `views.py` necesites mostrar la pantalla con la lista de productos, la ruta será extremadamente segura, clara y libre de errores:

```
return render(request, 'inventario/productos/lista.html')
```

3.5 El Enrutador (URLs) y la función `include`

Hasta ahora, hemos creado el diseño y la lógica de nuestro inventario, pero necesitamos que el "mesero" (el sistema de rutas) sepa cómo llegar a ellos.

Si pusiéramos todas las direcciones web de un proyecto grande en un solo archivo, sería un desastre. Para mantener la **modularidad**, Django nos permite delegar las rutas: el proyecto principal solo dirige el tráfico hacia las Apps, y cada App se encarga de sus propias rutas internas.

3.5.1 Configurar las rutas locales de la App (`inventario/urls.py`)

Primero, vamos a decirle al módulo de inventario cuáles son sus propias direcciones web.

1. Abre el archivo `urls.py` que creaste dentro de la carpeta `inventario`.
2. Escribe el siguiente código para enlazar la vista del dashboard:

```
from django.urls import path
from . import views
# El punto significa: "importa las vistas de esta misma carpeta"

# app_name nos ayuda a evitar conflictos de rutas entre diferentes
módulos
app_name = 'inventario'

urlpatterns = [
    # Esta es la ruta principal del inventario
    path('', views.dashboard_inventario, name='dashboard'),
]
```

3.5.2 Conectar la App al Proyecto Principal (*core/urls.py*)

Ahora, debemos ir al archivo central de rutas de nuestra "casa" e indicarle que todas las peticiones que busquen el inventario deben ser enviadas a nuestro nuevo módulo.

1. Abre el archivo `urls.py` que está dentro de la carpeta de configuración

`core.`

2. Importa la función `include` y agrega la ruta para tu aplicación:

```
from django.contrib import admin
from django.urls import path, include #¡Paso vital! Importamos include

urlpatterns = [
    path('admin/', admin.site.urls),

    # Le decimos a Django: "Si la URL empiece con 'inventario/',
    # envía al usuario al archivo urls.py de la app inventario"
    path('inventario/', include('inventario.urls')),
]
```

 **Nota técnica:** Al usar `include`, si el día de mañana el módulo de inventario

crece y tiene 50 pantallas diferentes, no tendrás que tocar el archivo principal del `core` nunca más. Toda la gestión se hará internamente en el `urls.py` de la app. ¡Eso es verdadero desarrollo modular!

3.6 Preparando la Interfaz (Herencia Dual y Bootstrap)

En un sistema de gestión empresarial, los usuarios tienen roles muy distintos.

Por ello, no usaremos una sola plantilla, sino dos "Plantillas Maestras":

1. `base_cliente.html`: Para la parte pública (Landing page, catálogo público, login).
2. `base_admin.html`: Para la parte privada o administrativa (Dashboard, inventario, reportes).

Para que nuestro software se vea profesional y sea *Responsive* (adaptable a celulares), utilizaremos **Bootstrap 5** a través de su CDN (enlaces directos a sus servidores).

3.6.1 Crear la Plantilla Pública (*base_cliente.html*)

1. Ve a la carpeta global `templates` (en la raíz de tu proyecto).
2. Crea el archivo `base_cliente.html` y escribe lo siguiente:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>{% block title %}Mi Empresa{% endblock title %}</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.
min.css" rel="stylesheet">
</head>
<body class="bg-light">

  <nav class="navbar navbar-expand-lg navbar-dark bg-primary">
    <div class="container">
      <a class="navbar-brand" href="/">Mi Proyecto ADSO</a>
      <div class="collapse navbar-collapse">
        <ul class="navbar-nav ms-auto">
          <li class="nav-item"><a class="nav-link"
href="/">Inicio</a></li>
          <li class="nav-item"><a class="nav-link"
href="#">Catálogo</a></li>
          <li class="nav-item"><a class="btn btn-outline-
light ms-2" href="/inventario/">Ingresar</a></li>
        </ul>
      </div>
    </div>
  </nav>

  <main class="container mt-4">
    {% block content %}
    {% endblock content %}
  </main>

  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bu
ndle.min.js"></script>
</body>
</html>
```

3.6.2 Crear la Plantilla Administrativa (*base_admin.html*)

Ahora crearemos el cascarón para los módulos internos, como el inventario. Este suele tener un diseño de "Dashboard" (panel de control).

1. En la misma carpeta `templates`, crea el archivo `base_admin.html`:

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Panel Administrativo - ADSO</title>
  <link
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min
.css" rel="stylesheet">
</head>
<body>
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">
      <a class="navbar-brand" href="#">ADSO AdminPanel</a>
      <span class="navbar-text text-white">
        Módulo de Gestión
      </span>
    </div>
  </nav>

  <div class="container-fluid">
    <div class="row">
      <nav class="col-md-3 col-lg-2 d-md-block bg-light sidebar py-
3" style="min-height: 90vh;">
        <ul class="nav flex-column">
          <li class="nav-item">
            <a class="nav-link active text-dark fw-bold"
href="/inventario/">📦 Inventario</a>
          </li>
          <li class="nav-item">
            <a class="nav-link text-secondary" href="#">👤
Usuarios</a>
          </li>
          <li class="nav-item">
            <a class="nav-link text-secondary" href="#">📊
Reportes</a>
          </li>
          <li class="nav-item mt-4">
            <a class="btn btn-sm btn-danger w-100"
href="/">Cerrar Sesión</a>
          </li>
        </ul>
      </nav>

      <main class="col-md-9 ms-sm-auto col-lg-10 px-md-4 mt-4">
        {% block content %}
        {% endblock content %}
      </main>
    </div>
  </div>

  <script
src="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundl
e.min.js"></script>
</body>
</html>

```

3.7 Crear el Template del Inventario (Usando Clases de Bootstrap)

Ahora vamos al módulo de inventario y le diremos que herede el diseño administrativo y usemos las tarjetas (*cards*) de Bootstrap para que se vea profesional.

1. Ve a `inventario/templates/inventario/index.html`.
2. Reemplaza el contenido por esto:

```
{% extends 'base_admin.html' %}

{% block content %}

    <div class="d-flex justify-content-between align-items-center mb-4">
        <h1 class="h2">Dashboard de Inventario</h1>
        <button class="btn btn-success">+ Registrar Producto</button>
    </div>

    <div class="row">
        <div class="col-md-4">
            <div class="card text-white bg-primary mb-3">
                <div class="card-body">
                    <h5 class="card-title">Total Productos</h5>
                    <p class="card-text display-6">0</p>
                </div>
            </div>
        </div>
        <div class="col-md-4">
            <div class="card text-white bg-warning mb-3">
                <div class="card-body">
                    <h5 class="card-title">Bajo Stock</h5>
                    <p class="card-text display-6">0</p>
                </div>
            </div>
        </div>
    </div>

{% endblock content %}
```

3.8 La Lógica (Nuestra primera Función en la Vista)

Esta parte es el corazón de nuestra aplicación. El "Chef" (la vista) no solo se encarga de despachar el HTML, sino que también puede enviarle información dinámica para que la página cambie dependiendo de lo que ocurra en el servidor.

A este paquete de información lo llamamos **Contexto (context)**. Funciona como un diccionario de Python que actúa como mensajero entre el Backend y el Frontend.

1. Abre el archivo `inventario/views.py`.
2. Escribe la siguiente lógica, agregando nuestro diccionario de contexto con un título dinámico:

```
from django.shortcuts import render

def dashboard_inventario(request):
    # 1. Preparamos los datos que queremos enviar a la interfaz
    contexto = {
        'titulo': 'Panel Central de Inventario',
        # Más adelante, aquí enviaremos datos de la base de datos,
        # por ejemplo: 'total_productos': 150
    }

    # 2. La vista carga el index, y le pasa el 'contexto' como tercer
    # parámetro
    return render(request, 'inventario/index.html', contexto)
```

3.8.1 Concepto Avanzado: Funciones vs. Clases

Antes de escribir nuestro código en el archivo `views.py`, debemos tomar una decisión arquitectónica. En Django, existen dos maneras de construir una vista (el "Chef" que procesa la petición): **Vistas Basadas en Funciones (FBV)** y **Vistas Basadas en Clases (CBV)**.

Como analistas y desarrolladores, debemos conocer ambas y saber cuándo usar cada una.

Característica	Vistas Basadas en Funciones (FBV)	Vistas Basadas en Clases (CBV)
¿Qué es?	Usamos <code>def</code> <code>nombre_vista(request) :.</code> Es el estilo tradicional de Python.	Usamos <code>class</code> <code>NombreVista(View) :.</code> Aprovecha la Programación Orientada a Objetos (POO).
Pros (Ventajas)	- Fáciles de leer y entender paso a paso. - Ideales para lógicas personalizadas muy complejas. - Excelentes para quienes están aprendiendo el ciclo de Django.	- Evitan repetir código (DRY). - Django ya trae clases listas para operaciones CRUD (Crear, Leer, Actualizar, Borrar). - Código mucho más limpio y corto.
Contras (Desventajas)	- Si tienes 5 módulos que hacen un CRUD, terminarás escribiendo casi el mismo código 5 veces. - Los archivos <code>views.py</code> pueden volverse gigantes.	- La curva de aprendizaje es más empinada (requiere dominio de POO y herencia). - Si necesitas hacer algo muy "fuera de lo común", puede ser complejo modificar la clase base.

💡 **Decisión para nuestro proyecto:** Para esta etapa de construcción, **comenzaremos utilizando Funciones (FBV)**. Esto nos permitirá entender exactamente cómo viaja la información (`request` y `response`). Más adelante, cuando vayamos a construir los mantenimientos (CRUD) de nuestros productos, migraremos a **Clases (CBV)** para agilizar el desarrollo.

3.8.2 La Anatomía de la Vista y la Introducción al ORM

Acabamos de escribir nuestra primera función en el archivo `views.py`, pero ¿qué está pasando realmente por debajo? Vamos a desglosarla para entender la teoría:

3.8.2.1 El parámetro *request* (La Petición):

Toda función de vista en Django **debe** recibir como primer parámetro el `request` (solicitud). Imagina que es un sobre o paquete que envía el navegador del usuario. Ese sobre contiene información vital:

1. ¿El usuario está desde un celular o un computador?
2. ¿Qué URL exacta solicitó?
3. ¿Envío datos de un formulario (como un nombre de usuario y contraseña)?
4. ¿Tiene una sesión activa (ya inició sesión)?

3.8.2.2 La función *render* (El Horno):

La palabra *render* significa "procesar" o "ensamblar". Lo que hace esta función es tomar tres ingredientes:

1. El `request` original.
2. El archivo HTML base (`inventario/index.html`).
3. El `contexto` (nuestras variables dinámicas). Luego, los mete a un "horno", los mezcla, y devuelve un archivo HTML puro y finalizado que el navegador del usuario puede entender y dibujar en pantalla. A esto se le llama **Response** (Respuesta).

3.8.2.3 ¿De dónde sacaremos los datos reales? (Introducción al ORM)

En nuestro ejemplo, escribimos el título `'Panel Central de Inventario'` a mano dentro del código. Pero en la vida real, el inventario cambiará todos los días. Necesitaremos ir a la base de datos para preguntar: "*¿Cuántos productos tenemos?*" o "*¿Cuáles están bajos de stock?*".

Aquí es donde entra la magia del **ORM (Mapeo Objeto-Relacional)**.

El **ORM** es el traductor oficial de Django. Normalmente, para hablar con una base de datos tendríamos que escribir código SQL puro. Por ejemplo, para buscar productos con bajo stock escribiríamos: `SELECT * FROM inventario_producto WHERE stock < 10;`

El problema del SQL puro es que si mañana cambiamos de base de datos (por ejemplo, de MySQL a PostgreSQL o Oracle), tendríamos que reescribir gran parte de nuestro código.

¿Qué hace el ORM por nosotros? Nos permite interactuar con la base de datos usando **solo código Python (Objetos)**. El ORM toma nuestro código Python y lo traduce automáticamente al lenguaje SQL exacto que necesite nuestra base de datos en ese momento.

Con el ORM de Django, la consulta anterior se escribiría así de fácil y en una sola línea: `Producto.objects.filter(stock__lt=10)`

3.9 El puente de comunicación: El Contexto y el Motor de Plantillas

Ya sabemos que la **Vista** es el "Chef", pero ¿cómo le entrega los ingredientes al "Plato" (Template)? Lo hace a través de un objeto llamado **Contexto**.

3.9.1 ¿Qué es el Contexto?

El contexto es un **diccionario de Python** que contiene todas las variables que queremos mostrar en nuestra página web. Cada "Clave" del diccionario se convierte en una variable que el HTML puede leer.

3.9.2 Sintaxis del Motor de Plantillas (DTL)

Django utiliza su propio lenguaje dentro del HTML llamado *Django Template Language* (DTL). Para usarlo, debemos memorizar estas dos reglas de oro:

1. **Variables** `{{ }}`: Se usan dobles llaves para **imprimir** o mostrar un valor en pantalla.

Ejemplo: `{{ titulo }}`

2. **Etiquetas** `{% %}`: Se usan llaves y porcentajes para la **lógica** (herencia, ciclos `for`, condicionales `if`).

Ejemplo: `{% extends 'base.html' %}` o `{% if productos %}`

3.9.3 Tabla: Uso de Variables y Lógica en el Template

Herramienta DTL	Sintaxis	Propósito en el Proyecto
Variables	<code>{{ nombre_variable }}</code>	Muestra datos simples: nombres, precios, títulos.
Ciclo For	<code>{% for item in lista %} ... {% endfor %}</code>	Se usa para recorrer la lista de productos del inventario y crear tablas dinámicas.
Condicional If	<code>{% if stock < 5 %} ... {% endif %}</code>	Permite mostrar alertas (ej: un mensaje en rojo si no hay existencias).
Filtros de Variable	<code>{{ precio floatformat:2 }}</code>	Permiten modificar la apariencia de un dato sin cambiar el valor real en la base de datos
Comentarios	<code>{# comentario #}</code>	Notas internas del programador que no se ven en el navegador.

3.9.4 Tabla: Filtros de Plantilla más utilizados en Proyectos Web

Filtro DTL	Ejemplo de Sintaxis	Resultado Visual	Uso Común en Inventario
lower	<code>{{ nombre lower }}</code>	tornillo hexagonal	Estandarizar textos para buscadores.
upper	<code>{{ categoria upper }}</code>	HERRAMIENTAS	Resaltar títulos o categorías principales.
capfirst	<code>{{ marca capfirst }}</code>	Stanley	Capitalizar la primera letra de una marca.
floatformat	<code>{{ precio floatformat:2 }}</code>	5000.50	Dar formato de dos decimales a precios y costos.
date	<code>{{ fecha date:"d/m/Y" }}</code>	06/04/2026	Mostrar fechas de vencimiento o registro de forma legible.
default	<code>{{ stock default:"Agotado" }}</code>	Agotado	Mostrar un mensaje amigable si un campo llega vacío de la base de datos.
truncatewords	<code>{{ desc truncatewords:5 }}</code>	Taladro percutor de alta...	Acortar descripciones largas en vistas de catálogo o tablas.
add	<code>{{ stock add:"10" }}</code>	25 (si stock era 15)	Realizar operaciones matemáticas rápidas en la interfaz.

💡 **Recomendación para el Aprendiz:** Recuerden que los filtros se pueden

"encadenar". Por ejemplo, si quieren que un nombre salga en mayúsculas y truncado, pueden usar: `{{ nombre|upper|truncatewords:2 }}`. Django procesará primero el primer filtro y el resultado lo pasará al segundo.

3.10 Creación de nuestro primer Modelo (Base de Datos)

Hasta ahora hemos preparado las vistas y los diseños, pero nuestro inventario necesita guardar información real. En lugar de abrir un gestor de base de datos para crear tablas a mano, utilizaremos el archivo `models.py` de nuestra aplicación.

1. Abre el archivo `models.py` ubicado dentro de tu carpeta `inventario`.
2. Vamos a crear dos entidades básicas usando Clases de Python:

Categoría y Producto. Borra el contenido por defecto y escribe el siguiente código:

```
from django.db import models

# 1. Tabla de Categorías
class Categoria(models.Model):
    nombre = models.CharField(max_length=100, unique=True)
    descripcion = models.TextField(blank=True, null=True)

    def __str__(self):
        return self.nombre

# 2. Tabla de Productos
class Producto(models.Model):
    nombre = models.CharField(max_length=150)
    precio = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.IntegerField(default=0)

    # Relación (Llave Foránea): Un producto pertenece a una categoría
    categoria = models.ForeignKey(Categoria, on_delete=models.CASCADE)

    fecha_registro = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.nombre} - Stock: {self.stock}"
```

💡 **Nota técnica:** La función `def __str__(self):` es una buena práctica obligatoria en Django. Le dice al sistema cómo debe mostrar ese objeto en texto cuando lo consultemos más adelante. En lugar de mostrar un genérico "Objeto 1", mostrará el nombre real del producto.

3.10.1 Concepto Clave: ¿Qué son los Modelos?

En la fase de análisis de nuestro proyecto, seguramente diseñaste un **Modelo Entidad-Relación (MER)** con tablas, atributos y conexiones. En Django, no creamos esas tablas escribiendo código SQL (como `CREATE TABLE...`); lo hacemos escribiendo **Clases de Python**. A estas clases las llamamos **Modelos**.

La traducción del diseño al código es directa:

- Una **Entidad** (o Tabla) del MER se convierte en una `class` de Python.
- Un **Atributo** (o Columna) del MER se convierte en una variable dentro de la clase (ej. `nombre = models.CharField(...)`).
- La **Llave Primaria (PK)** ¡Django la crea automáticamente! No necesitas escribir el campo `id`, el ORM lo hace por ti por defecto.

3.10.2 Entendiendo las Relaciones en Django

Las bases de datos relacionales se basan en conectar información. Veamos cómo Django maneja las dos relaciones más importantes para nuestro proyecto:

A. Relación Uno a Muchos (1:N) - `ForeignKey`

Es la más común. Por ejemplo: **Una Categoría tiene muchos Productos**, pero un Producto pertenece a una sola Categoría.

- **¿Dónde se pone el código?** La regla de oro es que la llave foránea *siempre* se coloca en la tabla que representa los "Muchos" (la tabla hija).
- **En el código:** Por eso, en nuestro modelo `Producto`, escribimos: `categoria = models.ForeignKey(Categoria, on_delete=models.PROTECT)`

El comportamiento de borrado (`on_delete`)

Cuando conectamos dos tablas, Django nos obliga a usar el parámetro `on_delete`. Esto le dice a la base de datos **qué debe hacer con los registros "hijos" si alguien elimina al registro "padre"**.

Opción de <code>on_delete</code>	¿Qué hace el sistema?	Ejemplo en nuestro Inventario	Cuándo usarlo
<code>models.CASCADE</code>	Efecto Dominó: Borra al padre y también a todos sus hijos asociados.	Se borra la categoría "Ferretería" Y se borran automáticamente los 50 productos que pertenecían a ella.	En relaciones de dependencia total. ¡Usar con cuidado!
<code>models.PROTECT</code>	Escudo: Bloquea el borrado del padre si este tiene hijos asociados, lanzando un error.	El sistema no te dejará borrar la categoría "Ferretería". Te pedirá que primero muevas o elimines los 50 productos.	Muy recomendado para sistemas de inventario para evitar borrar datos importantes por accidente.

<code>models.SET_NULL</code>	Dejar Huérfano: Borra al padre y deja a los hijos intactos, poniendo su campo en blanco (<code>NULL</code>).	Se borra "Ferretería". Los 50 productos se quedan en la base de datos, pero su categoría dirá "Ninguna".	Cuando no quieres perder el historial de los hijos. (<i>Exige <code>null=True</code></i>).
<code>models.SET_DEFAULT</code>	Asignar Sustituto: Borra al padre y le asigna a los hijos una categoría por defecto.	Se borra "Ferretería" y los 50 productos pasan a la categoría "Sin Clasificar".	(<i>Exige definir <code>default=...</code></i>).

B. Relación Muchos a Muchos (N:M) - `ManyToManyField`

Imagina que tenemos **Productos** y **Proveedores**. Un producto puede ser vendido por varios proveedores, y un proveedor puede vender varios productos.

- **El problema en SQL:** Estás obligado a crear una **tercera tabla** (tabla intermedia o pivote) para conectarlas.
- **La magia de Django:** No necesitas crear la **tabla intermedia**. Simplemente vas a uno de los dos modelos y agregas un campo `ManyToManyField`. Django creará la tabla intermedia en la base de datos de forma invisible y automática.

```
# Ejemplo de Muchos a Muchos (No copiar en el proyecto actual, solo ilustrativo)
class Proveedor(models.Model):
    nombre = models.CharField(max_length=100)

class Producto(models.Model):
    nombre = models.CharField(max_length=150)
    proveedores = models.ManyToManyField(Proveedor) # Magia automática
```

3.10.3 Tabla de Referencia: Tipos de Datos para el MER

Para traducir correctamente los atributos de tu MER a Django, utiliza esta "hoja de trucos":

Tipo de Dato en el MER	Código en Django (models.)	Uso principal en el Proyecto
Varchar / Texto Corto	<code>CharField(max_length=...)</code>	Nombres, marcas, códigos (SKU). Exige definir límite.
Texto Largo	<code>TextField()</code>	Descripciones detalladas, observaciones.
Entero (Int)	<code>IntegerField()</code>	Cantidades, stock, números de pasillo, edades.
Decimal / Float	<code>DecimalField(max_digits=..., decimal_places=...)</code>	Precios y costos. Vital para evitar pérdidas de centavos.
Booleano (Bit)	<code>BooleanField(default=True)</code>	Estados lógicos: ¿Está activo?, ¿Agotado?
Fecha (Date)	<code>DateField()</code>	Fechas estáticas: Fecha de vencimiento.
Fecha y Hora	<code>DateTimeField(auto_now_add=True)</code>	Auditoría: Fecha exacta en la que se creó un registro.
Foránea (1:N)	<code>ForeignKey(...)</code>	Para conectar tablas (ej. Producto -> Categoría).
Intermedia (N:M)	<code>ManyToManyField(...)</code>	Para relaciones múltiples.

💡 **Opciones adicionales importantes:** Dentro de los paréntesis puedes agregar reglas como `null=True` (permite celda vacía), `unique=True` (evita registros duplicados) o `default=0` (valor por defecto).

3.11 Preparando y aplicando Migraciones

Escribir el modelo en Python no crea la tabla mágicamente en la base de datos. Debemos usar las herramientas de Django para "traducir" ese código a SQL.
(Recuerda consultar la sección 5 al final de esta guía para repasar los comandos principales).

1. Asegúrate de estar en tu terminal, con el entorno virtual activado y en la raíz del proyecto.
2. **Paso 1 (Empaquetar los cambios):** Ejecuta el comando para que Django detecte tu nuevo código y prepare los archivos de traducción:

```
python manage.py makemigrations
```

Verás un mensaje verde en la consola indicando que se detectaron y crearon los modelos Categoría y Producto.
3. **Paso 2 (Impactar la Base de Datos):** Ahora, envía esos paquetes a la base de datos real con el comando:

```
python manage.py migrate
```

Resultado esperado: Verás una lista de "OKs" en la consola. ¡Felicidades!

Acabas de crear tus primeras tablas en la base de datos sin escribir una sola línea de SQL.

3.12 El Panel de Administración (Nuestra herramienta de Desarrollo)

Una de las características más potentes de Django es que nos regala un Panel de Administración (Backoffice) completamente funcional desde el minuto cero.

¿Para qué usaremos este panel en nuestro proyecto? Como desarrolladores, apenas estamos construyendo las interfaces gráficas de nuestro inventario. Mientras programamos las pantallas definitivas con Bootstrap para nuestro cliente, usaremos este panel interno para ingresar datos de prueba (categorías y productos) de forma rápida y segura, y así poder probar que nuestra base de datos funciona correctamente.

Para ingresar, este panel está bloqueado por seguridad. Necesitamos crear unas credenciales maestras y decirle explícitamente a Django qué tablas queremos ver allí.

3.12.1 Crear el Superusuario (El SysAdmin)

Vamos a crear el usuario administrador con permisos totales sobre el sistema.

1. Ve a tu terminal (asegúrate de que el entorno virtual `.venv` esté activo).
2. Ejecuta el siguiente comando: `python manage.py createsuperuser`
3. El sistema te hará una serie de preguntas. Llénelas de la siguiente manera:
 - **Username:** Escribe un nombre de usuario (ej. `admin`) y presiona *Enter*.
 - **Email address:** Puedes dejarlo en blanco o poner tu correo.
 - **Password:** Escribe una contraseña segura. **¡Atención!** Por medidas de seguridad en la consola, *no verás los caracteres mientras escribes*. Tú sigue escribiendo y presiona *Enter*.
 - **Password (again):** Repite la contraseña.

3.12.2 Registrar los Modelos en `admin.py`

Aunque ya creamos las tablas en la base de datos, el Panel de Administración es ciego por defecto. Hay que "registrar" las tablas para poder interactuar con ellas.

1. Abre el archivo `admin.py` que está dentro de la carpeta de tu app `inventario`.
2. Modifica el archivo para importar tus modelos y registrarlos:

```
from django.contrib import admin
from .models import Categoria, Producto

# Le decimos a Django que muestre estas tablas en el panel interno
admin.site.register(Categoria)
admin.site.register(Producto)
```

3.12.3 Poblar la Base de Datos (Primeros Datos de Prueba)

¡Es hora de ingresar nuestros primeros registros para usarlos más adelante en nuestras propias vistas!

1. Enciende el servidor ejecutando en la terminal: `python manage.py runserver`
2. Abre tu navegador web y ve a la ruta exclusiva del administrador:
`http://127.0.0.1:8000/admin`
3. Inicia sesión con el usuario y la contraseña que creaste.
4. **Misión Práctica:** Entra a la sección de "Categorías" y crea al menos 3 categorías (ej. *Herramientas*, *Materiales*, *Pinturas*). Luego, entra a "Productos" y registra 5 productos asignándoles las categorías que acabas de crear.

💡 **Visión a Futuro:** Los datos que acabas de ingresar nos servirán en la siguiente fase para mostrarlos en nuestro propio Dashboard personalizado (`inventario/index.html`). Más adelante, programaremos nuestros propios formularios (CRUD) para que el cliente final pueda registrar productos sin tener que entrar nunca a este panel de desarrolladores.

3.13 Conectando la Base de Datos con la Interfaz (Nuestra primera consulta)

Ya tenemos productos reales registrados gracias al Panel de Administración, pero nuestro `index.html` sigue mostrando datos falsos o "quemados" (como `Total Productos: 0`).

Vamos a decirle a nuestro "Chef" (la Vista) que vaya a la bodega (Base de Datos), cuente los productos reales y saque la lista para mostrársela al cliente.

1. **Actualizar la Vista (`views.py`):** Abre el archivo `inventario/views.py`. Vamos a importar nuestro modelo `Producto` y a usar una instrucción básica del ORM para traer todos los registros.

```
from django.shortcuts import render
# 1. Importamos los modelos que necesitamos leer
from .models import Producto, Categoria

def dashboard_inventario(request):
    # 2. Consultamos la base de datos (Traemos todos los productos)
    lista_productos = Producto.objects.all()

    # 3. Contamos cuántos productos hay en total
    total = lista_productos.count()

    # 4. Enviamos esos datos reales al contexto
    contexto = {
        'titulo': 'Dashboard Principal',
        'productos': lista_productos,
        'total_productos': total
    }

    return render(request, 'inventario/index.html', contexto)
```

2. **Imprimir los datos en el Template (`index.html`):** Ahora vamos a nuestro archivo `inventario/templates/inventario/index.html` y usamos las Etiquetas de Plantilla (que vimos en el paso 3.9) para reemplazar los ceros estáticos por los datos reales, y para crear una tabla dinámica.

```

{% extends 'base_admin.html' %}
{% block content %}

    <div class="d-flex justify-content-between align-items-center mb-4">
        <h1 class="h2">{{ titulo }}</h1>
    </div>

    <div class="row mb-4">
        <div class="col-md-4">
            <div class="card text-white bg-primary">
                <div class="card-body">
                    <h5 class="card-title">Total Productos</h5>
                    <p class="card-text display-6">{{ total_productos
}}</p>
                </div>
            </div>
        </div>
    </div>

    <h3>Lista de Productos</h3>
    <table class="table table-striped border">
        <thead class="table-dark">
            <tr>
                <th>Nombre</th>
                <th>Categoría</th>
                <th>Precio</th>
                <th>Stock</th>
            </tr>
        </thead>
        <tbody>
            {% for item in productos %}
            <tr>
                <td>{{ item.nombre }}</td>
                <td>{{ item.categoria.nombre }}</td>
                <td>${{ item.precio|floatformat:2 }}</td>
                <td>
                    {% if item.stock < 5 %}
                        <span class="badge bg-danger">{{ item.stock }}
(Bajo)</span>
                    {% else %}
                        <span class="badge bg-success">{{ item.stock
}}</span>
                    {% endif %}
                </td>
            </tr>
            {% empty %}
            <tr>
                <td colspan="4" class="text-center">No hay productos
registrados en la base de datos.</td>
            </tr>
            {% endfor %}
        </tbody>
    </table>

{% endblock content %}

```

Resultado: Al recargar tu página (<http://127.0.0.1:8000/inventario/>), verás cómo la tabla se llena automáticamente con la información que tú mismo registraste hace unos momentos en el panel de administrador. ¡Felicidades! Acabas de completar el ciclo fundamental de Django (Modelo-Vista-Template) y tu base de datos ya está conectada con tu interfaz.

4 Anexo: ¿Qué es el ORM?

Antes de crear nuestra primera tabla de base de datos, debemos entender cómo Django se comunica con ella. Para esto, el framework utiliza una tecnología llamada **ORM (Mapeo Objeto-Relacional)**.

Normalmente, para guardar o buscar información, un desarrollador tendría que escribir código en lenguaje SQL puro. Por ejemplo, para buscar los productos que tienen poco stock, escribiría algo así: `SELECT * FROM inventario_producto WHERE stock < 10;`

El problema del SQL puro es que si el día de mañana la empresa decide cambiar su base de datos (por ejemplo, pasar de MySQL a Oracle), tendríamos que revisar y reescribir gran parte de nuestras consultas manuales.

¿Qué hace el ORM por nosotros? El ORM actúa como un "traductor automático" de alto nivel. Nos permite interactuar con la base de datos utilizando **únicamente código Python (mediante Clases y Objetos)**. Él se encarga de tomar nuestro código Python y traducirlo al lenguaje SQL exacto que necesite el motor de base de datos que estemos usando.

Con el ORM de Django, la consulta anterior se escribiría en Python de esta manera: `Producto.objects.filter(stock__lt=10)`

Ventajas del ORM para nuestro software:

1. **Seguridad Integrada:** Nos protege automáticamente contra ataques informáticos muy comunes y peligrosos, como la "Inyección SQL", ya que el ORM limpia ("sanitiza") los datos antes de enviarlos a la base de datos.
2. **Agnosticismo de Base de Datos:** Podemos desarrollar el proyecto en nuestras computadoras usando una base de datos ligera que viene por

defecto (SQLite). Luego, cuando subamos el sistema a internet, podemos conectarlo a un motor robusto (como PostgreSQL o MySQL) **¡sin cambiar ni una sola línea de nuestro código Python!**

3. **Productividad y Código Limpio:** Escribimos mucho menos código. Al no tener cadenas de texto SQL mezcladas con nuestro Python, el proyecto es mucho más fácil de leer, mantener y escalar, permitiéndonos enfocarnos 100% en la lógica del negocio.

4.1 Filtros Avanzados (El uso del Doble Guion Bajo __)

Cuando usamos el ORM para buscar datos, a veces necesitamos algo más que una coincidencia exacta. Necesitamos buscar productos que sean *mayores que* cierto precio, o nombres que *contengan* cierta palabra.

Dado que en Python no podemos escribir algo como

`Producto.objects.filter(stock < 10)` (porque la sintaxis de la función no lo permite),

Django utiliza una técnica llamada **Field Lookups**, que consiste en agregar un **doble guion bajo (__)** seguido del tipo de filtro que queremos aplicar.

A continuación, presentamos la tabla con los sufijos más utilizados en la construcción de nuestro software, usando nuestro módulo de inventario como ejemplo:

Sufijo (Doble Guion Bajo)	¿Qué significa? (En inglés / Español)	Ejemplo de código en Django (ORM)	Equivalente en SQL
<code>__exact</code>	<i>Exact</i> : Coincidencia exacta (es el comportamiento por defecto si no ponemos nada).	<code>Producto.objects.filter(nombre__exact='Martillo')</code>	<code>WHERE nombre = 'Martillo'</code>
<code>__icontains</code>	<i>Ignore Case Contains</i> : Contiene un texto, sin importar si está en mayúsculas o minúsculas.	<code>Producto.objects.filter(descripcion__icontains='madera')</code>	<code>WHERE descripcion ILIKE '%madera%'</code>
<code>__gt</code>	<i>Greater Than</i> : Estrictamente Mayor que (>).	<code>Producto.objects.filter(precio__gt=50000)</code>	<code>WHERE precio > 50000</code>
<code>__gte</code>	<i>Greater Than or Equal</i> : Mayor o igual que (>=).	<code>Producto.objects.filter(precio__gte=50000)</code>	<code>WHERE precio >= 50000</code>
<code>__lt</code>	<i>Less Than</i> : Estrictamente Menor que (<).	<code>Producto.objects.filter(stock__lt=10)</code>	<code>WHERE stock < 10</code>
<code>__lte</code>	<i>Less Than or Equal</i> : Menor o igual que (<=).	<code>Producto.objects.filter(stock__lte=10)</code>	<code>WHERE stock <= 10</code>
<code>__startswith</code>	<i>Starts With</i> : El campo empieza exactamente con ese texto.	<code>Categoria.objects.filter(nombre__startswith='Herra')</code>	<code>WHERE nombre LIKE 'Herra%'</code>
<code>__in</code>	<i>In</i> : El valor está dentro de una lista específica.	<code>Producto.objects.filter(categoria_id__in=[1, 3, 5])</code>	<code>WHERE categoria_id IN (1, 3, 5)</code>

💡 **Tip de Desarrollo:** La letra **i** al principio de un filtro (como en `__icontains` o `__iexact`) significa **"Insensitive"**

(Insensible). Esto le dice a la base de datos que ignore si el usuario escribió en mayúsculas o minúsculas al hacer la búsqueda, lo cual es fundamental para los buscadores de nuestras aplicaciones web.

4.2 Operaciones Básicas con el ORM (El famoso CRUD)

En el desarrollo de software, el 90% del tiempo estaremos realizando cuatro acciones fundamentales sobre nuestra base de datos: **Crear**, **Read** (Leer), **Update** (Actualizar) y **Delete** (Borrar). A esto se le conoce mundialmente como el **CRUD**.

Para que veamos el verdadero poder y simplicidad del ORM de Django en nuestro módulo de inventario, aquí tienes una tabla con los ejemplos de código que más utilizarás a partir de ahora:

Operación (CRUD)	Ejemplo de Código en Django (Python)	Equivalente en SQL Tradicional	¿Qué hace exactamente?
Leer todos (Read)	<code>Producto.objects.all()</code>	<code>SELECT * FROM producto;</code>	Trae un listado con absolutamente todos los registros de esa tabla.
Leer uno solo (Read)	<code>Producto.objects.get(id=5)</code>	<code>SELECT * FROM producto WHERE id=5 LIMIT 1;</code>	Busca un registro único . Es ideal para buscar por ID. <i>(Nota: Si el producto no existe, Django lanzará un error para proteger el sistema).</i>
Filtrar varios (Read)	<code>Producto.objects.filter(estado='Activo')</code>	<code>SELECT * FROM producto WHERE estado='Activo';</code>	Trae una lista (QuerySet) solo con los registros que cumplan con la condición indicada.
Crear (Create)	<code>Producto.objects.create(nombre='Clavo', precio=200)</code>	<code>INSERT INTO producto (nombre, precio) VALUES ('Clavo', 200);</code>	Crea un nuevo registro y lo guarda automáticamente en la base de datos en un solo paso.
Actualizar (Update)	<code>item = Producto.objects.get(id=5)</code> <code>item.precio = 300</code> <code>item.save()</code>	<code>UPDATE producto SET precio=300 WHERE id=5;</code>	Primero traemos el objeto, modificamos su atributo en memoria, y finalmente usamos <code>.save()</code> para impactar la base de datos.
Borrar (Delete)	<code>item = Producto.objects.get(id=5)</code> <code>item.delete()</code>	<code>DELETE FROM producto WHERE id=5;</code>	Selecciona un registro específico y lo elimina permanentemente del sistema.

💡 **Tip de Rendimiento:** Observa que en Django no escribimos consultas

largas ni nos preocupamos por la sintaxis de la base de datos. Además, métodos como `.filter()` y `.all()` son "perezosos" (*lazy*); esto significa que Django no saturará el servidor haciendo la consulta a la base de datos hasta el momento exacto en que necesites mostrar esos datos en el HTML.

4.3 Filtros con Múltiples Condiciones (AND / OR)

En el mundo real, las consultas no siempre son de un solo campo. A veces el jefe de bodega nos dirá: "Necesito los productos que sean de la categoría 'Herramientas' **Y** que además tengan menos de 5 unidades" o "Tráeme los productos que sean de 'Pinturas' **O** de 'Solventes'".

Para esto, el ORM de Django maneja dos lógicas:

4.3.1 El operador "Y" (AND)

En Django, para aplicar un **AND**, simplemente separamos los filtros con una **coma (,)** dentro del mismo `.filter()`. El sistema solo devolverá los registros que cumplan **todas** las condiciones.

4.3.2 El operador "O" (OR)

Para aplicar un **OR**, Django utiliza una herramienta llamada **Clases Q**. Esto nos permite agrupar condiciones y separarlas con el símbolo de la barra vertical (`|`).

4.3.3 Operadores Lógicos en el ORM

Esta tabla ayudará a los aprendices a decidir cómo estructurar sus consultas complejas:

Operador Lógico	Sintaxis en Django ORM	Ejemplo Práctico (Inventario)	¿Qué le estamos diciendo a la Base de Datos?
AND (Y)	Separar por coma (,)	<code>Producto.objects.filter(categoria_id=1, precio__lt=5000)</code>	"Tráeme los productos que sean de la categoría 1 Y ADEMÁS su precio sea menor a 5000".
OR (O)	Símbolo Pipe () con <code>Q()</code>	<code>Producto.objects.filter(Q(stock=0) Q(estado='Agotado'))</code>	"Tráeme los productos que no tengan existencias O que tengan el estado marcado como Agotado".
NOT (NO)	Símbolo Virgulilla (~) con <code>Q()</code>	<code>Producto.objects.filter(~Q(categoria_id=5))</code>	"Tráeme todos los productos de la tabla EXCEPTO los que pertenecen a la categoría 5".
Combina do	Mezcla de , y	<code>filter(Q(cat=1) Q(cat=2), stock__gt=0)</code>	"Tráeme productos de la categoría 1 o 2, PERO SIEMPRE QUE tengan stock disponible".

5 Anexo: Comandos Principales de Django

A continuación, se presenta una tabla de referencia rápida con los comandos que utilizarás a diario para construir, administrar y probar tu software a través del archivo `manage.py`:

#	Comando	¿Para qué se usa? (Propósito)
1	<code>pip install django</code>	Descarga e instala el framework Django en tu entorno activo.
2	<code>django-admin startproject core .</code>	Genera la estructura base del proyecto. El punto (.) al final es crucial para que se construya en la carpeta actual sin redundancias.
3	<code>python manage.py startapp <nombre_app></code>	Crea un nuevo módulo o "aplicación" dentro del proyecto (por ejemplo: <code>python manage.py startapp usuarios</code> o <code>inventario</code>).
4	<code>python manage.py makemigrations</code>	Empaqueta los cambios que hayas hecho en tus modelos (las estructuras de tus datos) y los prepara para ser enviados a la base de datos.
5	<code>python manage.py migrate</code>	Ejecuta las migraciones pendientes. Es decir, crea o actualiza las tablas reales en la base de datos basándose en tus modelos.
6	<code>python manage.py createsuperuser</code>	Crea un usuario administrador con todos los permisos para poder acceder al panel de control (Admin) que Django trae por defecto.
7	<code>python manage.py runserver</code>	Inicia el servidor de desarrollo local. Te permite ver y probar tu aplicación en el navegador (generalmente en <code>http://127.0.0.1:8000/</code>).
8	<code>python manage.py shell</code>	Abre una consola interactiva de Python conectada a tu proyecto, muy útil para probar consultas a la base de datos o depurar código.

6 Anexo: Solución a Errores Frecuentes (Troubleshooting)

En el desarrollo de software, los errores (Bugs) no son fracasos, son el sistema dándonos pistas de lo que falta. A continuación, presentamos los errores más comunes al iniciar con Django y cómo solucionarlos rápidamente.

6.1 Error 1: "It is impossible to add a non-nullable field... without specifying a default"

- **¿Cuándo pasa?** Al ejecutar `python manage.py makemigrations` después de agregar un nuevo campo obligatorio a un modelo que ya tenía datos guardados previamente.
- **¿Qué significa?** La base de datos te está preguntando: *"Tengo registros viejos, ¿qué dato le pongo en esta columna nueva a esos registros que ya existían?"*.
- **La Solución:**
 1. En la terminal, elige la opción **2 (Quit)** para cancelar la migración de forma segura.
 2. Ve a tu `models.py` y agrégle un valor por defecto o permite nulos al nuevo campo.

Ejemplo: `descuento = models.IntegerField(default=0)` **0**

`descuento = models.IntegerField(null=True)`

3. Vuelve a ejecutar el comando `makemigrations`.

6.2 Error 2: `TemplateDoesNotExist` at `/ruta/`

- **¿Cuándo pasa?** Al abrir una página en el navegador después de configurar una Vista.

- **¿Qué significa?** Django llegó a tu "Chef" (la Vista), pero cuando este intentó buscar el "Plato" (el archivo HTML), no lo encontró en la ruta especificada.
- **La Solución (Revisa estas 3 cosas):**
 1. **En la Vista:** Revisa que la ruta en el `render` esté bien escrita. (Ej. `inventario/index.html`).
 2. **En las Carpetas:** Verifica que tu archivo HTML realmente esté guardado en `tu_app/templates/tu_app/index.html`.
 3. **En settings.py:** Asegúrate de haber registrado el nombre de tu aplicación en la lista `INSTALLED_APPS` y que la variable `TEMPLATES` tenga configurado `BASE_DIR / 'templates'`.

6.3 Error 3: `OperationalError: no such table: inventario_producto`

- **¿Cuándo pasa?** Al intentar entrar al Panel de Administración o cargar una página que consulta datos.
- **¿Qué significa?** Creaste tu modelo en Python (`models.py`), pero se te olvidó enviarle esos planos a la base de datos real.
- **La Solución:**
 1. Detén el servidor en la consola (Ctrl + C).
 2. Ejecuta: `python manage.py makemigrations`
 3. Ejecuta: `python manage.py migrate`
 4. Vuelve a encender el servidor con `runserver`.

6.4 Error 4: `ModuleNotFoundError: No module named 'django'`

- **¿Cuándo pasa?** Al intentar ejecutar cualquier comando con `python manage.py...` en la terminal.
- **¿Qué significa?** Tu terminal no reconoce a Django porque **olvidaste activar tu entorno virtual**. Estás usando el Python global de tu computador, que está vacío.
- **La Solución:**
 1. Ve a tu gestor de entornos en VS Code (Python Environments).
 2. Activa tu entorno `.venv` (asegúrate de ver el `(.venv)` al inicio de la línea de tu terminal).
 3. Vuelve a intentar tu comando.